Lesson 5 of 5

Score, crashing and reset

Add a score and a clean restart to finish the game.

Goal	A score display, and a clean restart when the snake hits a wall or itself.
Concepts	global, avoiding repeated code, any(), text on screen.
Starting point	Lesson 4's finished code. The complete game is on the last page.
Teacher prep	Optionally, show the version with copy-pasted reset code so students can spot the repetition.

Session plan (30 minutes)

Time	Activity
0-5	Run Lesson 4. Ask: what happens when you crash? How do you know how well you did?
5-20	Add the scoreboard, update_score(), reset(), and the new game_loop.
20-27	Test it. Crash on purpose into walls and into the tail.
27-30	Pick a challenge and celebrate the finished game.

Step 1: score variables

Add these **after the segments list**:

```
score = 0

scoreboard = turtle.Turtle()
scoreboard.color("white")
scoreboard.penup()
scoreboard.hideturtle()
scoreboard.goto(0, 260)
```

Step 2: two new functions

Add these **above game_loop**:

```
def update_score():
    scoreboard.clear()
    scoreboard.write(f"Score: {score}", align="center",
                     font=("Arial", 18, "normal"))

def reset():
    global score
    head.goto(0, 0)
    head.direction = "stop"
    for seg in segments:
        seg.hideturtle()
    segments.clear()
    score = 0
    update_score()
```

Step 3: replace game_loop

```
def game_loop():
    global score
```

```
if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
    reset()

if head.distance(food) < STEP:
    move_food()
    add_segment()
    score += 1
    update_score()

move_body()
move_head()

if any(seg.distance(head) < STEP for seg in segments):
    reset()

screen.update()
screen.ontimer(game_loop, 100)
```

Step 4: show the score from the start

Call **update_score()** once just before **game_loop()** at the bottom of the file.

Teaching notes

Ask students to find the repeated code they *would* have written (a reset for the wall and another for the tail), then show how one **reset()** replaces both. For **global**: without it, Python creates a brand-new score inside the function instead of changing the real one. For **any()**: read it aloud as "is any segment touching the head?"

Challenges (pick one)

- Speed the game up a little every time the snake eats.
- Add a high score that survives a reset.
- Add a second food worth 5 points that disappears after a few seconds.

Common problems

What you see	Likely cause and fix
Score stays at 0 or shows wrong numbers	**global score** is missing in game_loop or reset.
Score text overlaps itself	scoreboard.clear() is missing from update_score().
Old tail segments stay on screen after a crash	The seg.hideturtle() loop is missing from reset().
Snake resets instantly after eating	The tail may be overlapping the head. Check that move_body() runs before move_head().

What we learned (student checklist)

[ ] I can show text on screen and update it.

[ ] I can use global to change a variable from inside a function.

[ ] I can replace repeated code with one function.

[ ] I built a complete game!

Complete game (final code)

```
import turtle
import random

STEP = 20
LIMIT = 280

screen = turtle.Screen()
screen.title("Snake")
screen.bgcolor("black")
screen.setup(600, 600)
screen.tracer(0)

head = turtle.Turtle("square")
head.color("lime")
head.penup()
head.direction = "stop"

food = turtle.Turtle("circle")
food.color("red")
food.penup()
food.goto(0, 100)

segments = []
score = 0

scoreboard = turtle.Turtle()
scoreboard.color("white")
scoreboard.penup()
scoreboard.hideturtle()
scoreboard.goto(0, 260)

def go_up():
    if head.direction != "down": head.direction = "up"
def go_down():
    if head.direction != "up": head.direction = "down"
def go_left():
    if head.direction != "right": head.direction = "left"
def go_right():
    if head.direction != "left": head.direction = "right"

def move_head():
    if head.direction == "up":    head.sety(head.ycor() + STEP)
    if head.direction == "down":  head.sety(head.ycor() - STEP)
    if head.direction == "left":  head.setx(head.xcor() - STEP)
    if head.direction == "right": head.setx(head.xcor() + STEP)

def move_body():
    for i in range(len(segments) - 1, 0, -1):
        segments[i].goto(segments[i - 1].pos())
    if segments:
        segments[0].goto(head.pos())

def add_segment():
    seg = turtle.Turtle("square")
    seg.color("green")
    seg.penup()
    segments.append(seg)

def move_food():
    food.goto(random.randrange(-LIMIT, LIMIT, STEP),
              random.randrange(-LIMIT, LIMIT, STEP))

def update_score():
    scoreboard.clear()
```

```
    scoreboard.write(f"Score: {score}", align="center",
                     font=("Arial", 18, "normal"))

def reset():
    global score
    head.goto(0, 0)
    head.direction = "stop"
    for seg in segments:
        seg.hideturtle()
    segments.clear()
    score = 0
    update_score()

def game_loop():
    global score

    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
        reset()

    if head.distance(food) < STEP:
        move_food()
        add_segment()
        score += 1
        update_score()

    move_body()
    move_head()

    if any(seg.distance(head) < STEP for seg in segments):
        reset()

    screen.update()
    screen.ontimer(game_loop, 100)

screen.listen()
screen.onkeypress(go_up, "Up")
screen.onkeypress(go_down, "Down")
screen.onkeypress(go_left, "Left")
screen.onkeypress(go_right, "Right")

update_score()
game_loop()
screen.mainloop()
```